Technical Documentation: CarLog Project

Web Application Development - Javier Ramírez Serrano

April 2026

Contents

1 General Backend Analysis **2**
1.1 Server Technology Stack 2

2 Layered Architecture **2**
2.1 Controller Layer (API Layer) 2
2.2 Service Layer (Business Logic) 2
2.3 Persistence Layer 2

3 Security and Authentication System **2**
3.1 Stateless Authentication Flow 3
3.2 Client-Side Security (Angular Guards) 3

4 Domain Entities **3**

5 Deployment and DevOps **3**

6 Real-Time Communication System (WebSockets) **4**
6.1 Infrastructure and Messaging (Backend) 4
6.2 Reactivity and Event Bus (Frontend) 4
6.3 Critical Use Cases 5

7 Advanced Design Features **5**

8 Centralized Error Handling and Visual Feedback **5**
8.1 Backend: Global Exception Handler 5
8.2 Frontend: Interceptors and SweetAlert2 5

1 General Backend Analysis

The CarLog backend ecosystem is built on a next-generation stack, prioritizing stateless security and multi-workshop scalability.

1.1 Server Technology Stack

- **Framework:** Spring Boot 4.0.1.
- **Language:** Java 21 (LTS).
- **Persistence:** Spring Data JPA with MySQL 8.0.
- **Security:** Spring Security with JWT (JSON Web Tokens).
- **Infrastructure:** Docker & Docker Compose.

2 Layered Architecture

The system follows a design pattern that separates presentation logic from pure business logic.

2.1 Controller Layer (API Layer)

Controllers act as orchestrators for REST requests. They implement granular validations and transform entities into DTOs.

- `VehicleController`: CRUD operations and entry/exit flows.
- `UserController`: Client and employee management.
- `WorkOrderController`: Work order and service line management.
- `WorkshopController`: Employee administration and workshop profile management.

2.2 Service Layer (Business Logic)

This is the core of the system, where business rules are applied and transactionality is managed using the `@Transactional` annotation.

- **Financial Calculation:** The `WorkOrderService` automates the calculation of total amounts, dynamically processing work lines to break down the tax base, VAT quotas, and applied discounts.
- **File Management:** Services receive binary files via `MultipartFile` objects. The system processes these data streams for physical persistence in the server's file system (`/uploads` directory), storing only the necessary reference or metadata in the MySQL database.

2.3 Persistence Layer

Uses JPA repositories for SQL query abstraction. The system is designed for data isolation between workshops (Multi-tenancy).

3 Security and Authentication System

CarLog implements enterprise-level security based on JWT tokens.

3.1 Stateless Authentication Flow

Unlike traditional systems, the JWT is not sent in the response body. Instead, it is injected into an **HTTP-only Cookie** to mitigate security risks such as XSS.

Listing 1: Security Filter Configuration

```java
// JwtAuthenticationFilter.java
// Intercepts requests and extracts the JWT from the cookie
String token = getJwtFromCookie(request);
if (jwtService.isTokenValid(token, userDetails)) {
    SecurityContextHolder.getContext().setAuthentication(authToken);
}
```

3.2 Client-Side Security (Angular Guards)

In the presentation layer (Frontend), access control and route protection are managed through **Angular Guards**, ensuring users only access permitted views based on their session state and business role:

- `loginGuard`**:** Redirects users who already have an active session directly to the dashboard, preventing redundant access to the authentication screen.
- `authguardGuard`**:** Acts as the main barrier protecting all internal application routes, requiring a valid JWT token to exist in the *Storage* (Local or Session).
- `tallerGuard`**:** Implements specific business logic. Verifies that a user with a managerial role has a registered workshop before allowing access to workshop management tools, preventing data inconsistencies.
- `altaTallerGuard`**:** Controls access to the workshop creation form, blocking the route for users who already own a registered workshop to prevent duplicates.

4 Domain Entities

Entity	Critical Responsibility
`User`	RBAC Management (MANAGER, MECHANIC, CLIENT, DIY).
`Vehicle`	Stores technical specifications and workshop history.
`WorkOrder`	Repair header with states: PENDING, IN_PROGRESS, COMPLETED.
`Workshop`	Multi-tenant entity that isolates employees and vehicles.

5 Deployment and DevOps

The project includes native configuration for containerized environments, facilitating deployment in staging and production environments.

Listing 2: Docker Compose Structure

```yaml
services:
  mysql:
    image: mysql:8.0
    environment:
      MYSQL_DATABASE: carlog_db
  backend:
```

```
    build: .
    ports:
      - "8081:8081"
    depends_on:
      - mysql
```

6 Real-Time Communication System (WebSockets)

To ensure a smooth and reactive user experience, CarLog implements an asynchronous notification system based on the **STOMP** (*Simple Text Oriented Messaging Protocol*) protocol over **WebSockets**.

6.1 Infrastructure and Messaging (Backend)

The server acts as a message broker using `Spring WebSocket`. Communication is managed through the `SimpMessagingTemplate` component, which allows targeted message routing to specific topics.

- **Connection Endpoint:** `/ws-carlog/**` (Configured as public in `SecurityConfig` to allow the initial *handshake*).
- **Topic Structure:** A private topic architecture per user is used: `/topic/notificaciones/{dni}`.
- **Data Protocol:** Messages are encapsulated in the `NotificationDTO` object, consisting of: `type`, `title`, and `message`.

Listing 3: Notification Emission Examples in Spring Boot

```
// 1. Employment Invitation Flow
messagingTemplate.convertAndSend("/topic/notificaciones/" + employeeDni, notif);

// 2. Proposal Acceptance Flow
messagingTemplate.convertAndSend("/topic/notificaciones/" + managerDni, alert);

// 3. Vehicle Entry Request
messagingTemplate.convertAndSend("/topic/notificaciones/" + ownerDni, notif);
```

6.2 Reactivity and Event Bus (Frontend)

On the client side, the Angular application uses the `RxStomp` library to keep the connection active. To decouple message reception from component logic, a **Reactive Event Bus** is implemented via the `NotificationBusService`.

- **Event Types (`AppEventType`):** `RELOAD_VEHICLES`, `RELOAD_EMPLOYEES`, `NEW_INVITE`.
- **Signal Distribution:** Upon receiving a message through the socket, the `DashboardLayout` captures the notification and emits an event through the bus, triggering automatic data reloads in subscribed components without the need for page refreshes.

Listing 4: Event Subscription and Emission in Angular

```
this.rxStomp.watch(`/topic/notificaciones/${miDni}`).subscribe((message) => {
    const notification = JSON.parse(message.body);

    switch (notification.type) {
        case 'VEHICLE_REQUEST':
```

```
            this.mostrarToast(notification);
            this.notificationBus.emit(AppEventType.RELOAD_VEHICLES);
            break;
        case 'FIRE':
            this.authService.logout(); // Immediate logout due to termination
            break;
    }
});
```

6.3 Critical Use Cases

1. **Personnel Management:** Allows Managers to receive hiring confirmations instantly and notifies employees of their termination (`FIRE`), forcing a secure session logout.

2. **Owner Validation:** When a workshop requests a vehicle's entry, the owner receives an instant alert to authorize the operation from their "My Vehicles" panel, streamlining the workshop entry flow.

7 Advanced Design Features

- **DTO/Entity Separation:** Prevents the exposure of sensitive data and optimizes JSON responses.
- **WebSockets:** STOMP integration for reactive notifications (Invitations, entry alerts).
- **File Management:** Physical persistence in the `/uploads` folder with automated cleanup logic for orphaned files.

8 Centralized Error Handling and Visual Feedback

The system implements a robust architecture for error capture, processing, and presentation, ranging from database exceptions to the user interface, ensuring stability and an optimal user experience (UX).

8.1 Backend: Global Exception Handler

In the Spring Boot layer, the `@RestControllerAdvice` annotation is coupled with the `GlobalExceptionHandler` class to centralize exception handling across the API. Instead of returning native *stack traces* to the client (a practice that exposes vulnerabilities and breaks the client), this interceptor catches specific business exceptions (such as `VehicleNotFoundException`, `UserNotFoundException`, or `VehicleOcuppiedException`) and builds a standardized HTTP response. This error DTO always includes the validated exception message, the appropriate HTTP status code (404, 409, 500), and a *timestamp*.

8.2 Frontend: Interceptors and SweetAlert2

On the client side (Angular), error asymmetry is managed using an HTTP Interceptor (`error-interceptor.ts`). This global service catches the standardized error responses emitted by the `GlobalExceptionHandler`. Once the error is intercepted, the system prevents application crashing and extracts the clean message to provide asynchronous visual *feedback* using the **SweetAlert2** library. These modal alerts or *toast* notifications are designed consistently with the application's *Dark Theme* (using hex colors like `#212529`), providing clear information to the end-user (e.g., "The workshop already has access to your vehicle" or "Invalid credentials") without disrupting their navigation flow.